Verified Verification:
A Lean 4 Proof That the Metamath Checker Is Correct

Zarathustra Goertzel (supporting author and human supervisor) Oruži**

February 2026

Abstract

We present a formal verification, carried out entirely in Lean 4 with the Batteries library (no Mathlib), that the Metamath proof checker `checkBytes` correctly validates mathematical proofs. AI assistance: Codex, ChatGPT Pro, and Claude Code (under human supervision). The central result is an end-to-end *biconditional*: given a raw `ByteArray`, the parser accepts a proof for a formula f *if and only if* f is provable under the Metamath specification (`Spec.Provable`). This biconditional is further bridged to Mario Carneiro's independent declarative semantics (`Semantic.Provable`). Under a strict configuration (`prefixCertified`), we additionally prove *prefix provenance*: every accepted theorem is provable using only earlier theorems, ruling out circular reasoning. The development comprises 54,130 lines of Lean across 46 files, with zero `sorry`s, zero axioms, and a clean build of 129 modules. All 55 parser error codes carry certified semantic evidence. To our knowledge, this is the first complete soundness-and-completeness proof of a Metamath verifier implementation.

Contents

* "Oruži" denotes AI-assisted collaboration (Codex, ChatGPT Pro, Claude Code; mostly 5.* and 4.* series).

1 Introduction

Metamath [1] is a minimalist proof language used by a community of formalists to develop large databases of checked theorems, including `set.mm` (over 40,000 theorems in ZFC set theory). Its simplicity—constants, variables, substitution, disjoint-variable constraints, and a stack-based proof checker—makes it an attractive target for formal verification.

Several implementations of the Metamath proof checker exist: `metamath.exe` (C), `mmj2` (Java), `mm0-rs` (Rust), and Mario Carneiro's Lean 4 implementation `checkBytes` in `Verify.lean`. All of these have been extensively tested, but none has been *formally verified*: proved correct against a mathematical specification.

In this paper we close that gap. We prove, in Lean 4, that `checkBytes` is both *sound* (if it accepts a proof, the theorem is genuinely provable) and *complete* (if a theorem is provable, the checker will accept a proof for it). These two directions are combined into a single biconditional (§3).

Key results.

1. **End-to-end biconditional** (`verify_parser_acceptance_iff_spec_provable`): from raw `ByteArray` to `Spec.Provable`, for any configuration.

2. **Semantic bridge** (`verify_parser_acceptance_iff_semantic_provable_total`): the same biconditional bridged to Carneiro's declarative `Semantic.Provable`, using total database extraction.

3. **Prefix provenance** (`checkBytes_done_finishProofEvent_certified`): every accepted `$p` theorem is provable in the *pre-insertion* database, under a strict configuration.

4. **Mode equivalence** (`ProofReachableZ_iff_NormalProofReachable`): compressed and normal proof modes are equivalent at the database level.

5. **Error code certification** (`checkBytes_parseErrorCode?_fullyCertified`): all 55 error codes carry certified payload and rule-semantic violation evidence.

Trust boundary. The formal theorems cover `checkBytes` given a `ByteArray`. Include expansion (`$[ ... $]`) is treated as a pre-processing step outside the formalized core. The verified part is: given fully expanded bytes, the parser correctly checks every proof.

Organization. §2 maps the Metamath specification to Lean types. §3 presents the main contract theorems. §4 describes the generic spec-level bridge. §5 covers prefix provenance. §6 documents what is *not* claimed. §7 gives reproduction instructions. §8 describes the architecture and proof strategy. §9 concludes. Appendix A contains key theorem listings, Appendix B provides a file inventory, and Appendix C lists all unit tests grouped by specification rule.

2 Metamath Specification Anchors

The Metamath language is defined in Chapter 4 of the Metamath book [1]. We briefly recap the core concepts and their Lean formalizations.

2.1 Core Concepts

A Metamath database declares *constants* (fixed symbols like `wff`, `|-`, `(`) and *variables* (placeholders that can be substituted). An *expression* is a pair of a typecode (a constant) and a list of symbols. A *substitution* maps variables to expressions.

Disjoint variable (DV) constraints (`$d` statements) restrict which variables may share substituted variables—this is the mechanism that prevents unsound capture.

A *frame* collects the mandatory hypotheses (floating `$f` and essential `$e`) and DV constraints for an assertion. Table 1 shows the correspondence between specification sections and Lean types.

Table 1: Metamath specification sections mapped to Lean types.

Section	Topic	Lean type
§4.2.2	Constants and variables	`CN`, `VR`, `Sym`
§4.2.3	Expressions	`Expr`, `Expr.subst`
§4.2.4	Disjoint variable restrictions	`DJ`, `DJ.subst`
§4.2.5	Floating and essential hypotheses	`Formula`, `VR.vhyp`
§4.2.6	Assertions (`$a`, `$p`)	`Statement`
§4.2.7	Frames	`Context`
§4.3	Proof verification	`Provable`

2.2 Two Formalizations of Provability

We maintain two independent formalizations of what it means for a theorem to be provable:

Operational (`Spec.Provable`). Defined in `Spec/Operational.lean`, this models the Metamath proof checker as a stack machine. A proof is a sequence of steps; each step either pushes a hypothesis or applies an assertion (popping arguments and pushing the conclusion). A formula is provable if there exists a step sequence that produces a singleton stack containing it.

```
def Provable (G : Database) (fr : Frame) (e : Expr) : Prop :=
  exists (steps : List ProofStep) (finalStack : List Expr),
    ProofValid G fr finalStack steps
    and finalStack = [e]
```

Listing 1: Operational provability (`Spec/Operational.lean`:88–91).

The `ProofValid` inductive relation (Listing A.1 in Appendix A) defines the step-by-step stack machine execution, with constructors for essential hypotheses, floating hypotheses, and axiom/theorem application (including DV checking and type preservation).

Declarative (`Semantic.Provable`). Defined in `DeclarativeSpec.lean` by Mario Carneiro, this is a "big-step" inductive with three constructors: use a hypothesis directly, use a floating variable typing, or apply an axiom with a substitution that satisfies all DV constraints and whose hypothesis instantiations are all provable.

```
inductive Provable (axs : Statement -> Prop) (G : Context)
    : Formula -> Prop
  | hyp (h) : h in G.hyps -> Provable axs G h
  | var (v:VR) : v.vhyp in G.hyps -> Provable axs G v
  | ax (s) {ax} : axs ax -> ax.ctx.dj.subst s G.dj ->
    (forall h in ax.ctx.hyps, Provable axs G (h.subst s)) ->
    (forall v in ax.vars, Provable axs G (v.type, s v)) ->
    Provable axs G (ax.fmla.subst s)
```

Listing 2: Declarative provability (`DeclarativeSpec.lean`:463–472).

The operational and declarative formulations are proven equivalent in §4.

3 The Main Contract

The central theorem of the project is a biconditional connecting the *actual Lean implementation* (`checkBytes` in `Verify.lean`) to the *specification* (`Spec.Provable` in `Spec/Operational.lean`).

3.1 End-to-End Biconditional

```
theorem verify_parser_acceptance_iff_spec_provable
    (bytes : ByteArray)
    (label : String)
    (f : Verify.Formula)
    (h_success : (Verify.checkBytes bytes).error? = none) :
    (exists (proof : Array String) (pr_final : Verify.ProofState)
           (f' : Verify.Formula),
      proof.foldlM (fun pr step =>
        Verify.DB.stepNormal (Verify.checkBytes bytes) pr step)
        <..., label, f, (Verify.checkBytes bytes).frame,
             #[], #[], .normal> =
          Except.ok pr_final
      and pr_final.stack.size = 1
      and pr_final.stack[0]? = some f'
      and toExpr f' = toExpr f)
    <->
    (exists (G : Spec.Database) (fr : Spec.Frame),
      toDatabase (Verify.checkBytes bytes) = some G
      and toFrame (Verify.checkBytes bytes)
           (Verify.checkBytes bytes).frame = some fr
      and Spec.Provable G fr (toExpr f))
```

Listing 3: End-to-end biconditional (`KernelClean.lean`:10976).

Reading the theorem. The left-hand side says: there exists a proof array such that folding `stepNormal` over it (starting from an initial proof state with the given label and formula) succeeds, producing a singleton stack whose top element has the same `toExpr` image as `f`. The right-hand side says: the database and frame extracted from the parser state witness `Spec.Provable` for the `toExpr` image of `f`.

The only hypothesis is `h_success`: the parser completed without error. The theorem holds for *any* configuration.

3.2 Semantic Bridge

The biconditional is further bridged to Carneiro's declarative semantics:

```
theorem verify_parser_acceptance_iff_semantic_provable_total
    (bytes : ByteArray)
    (label : String)
    (f : Verify.Formula)
    (h_success : (Verify.checkBytes bytes).error? = none) :
    (exists (proof : Array String) (pr_final : Verify.ProofState)
           (f' : Verify.Formula),
      proof.foldlM (...) = Except.ok pr_final
      and pr_final.stack.size = 1
      and pr_final.stack[0]? = some f'
      and toExpr f' = toExpr f)
    <->
    (exists (fr : Spec.Frame),
      toFrame (Verify.checkBytes bytes)
           (Verify.checkBytes bytes).frame = some fr
      and Spec.Semantic.Provable
           (dbToAxioms (toDatabaseTotal (Verify.checkBytes bytes)))
           (frameToContext fr)
           (exprToFormula (varMapOfFrame fr) (toExpr f)))
```

Listing 4: Semantic bridge (`ParserEquivalence.lean`:441).

Note that the right-hand side uses `toDatabaseTotal`—a total (non-`Option`) database extraction. The `Option` wrapper in the original `toDatabase` was always `some` by construction; `toDatabaseTotal` makes this explicit.

3.3 Mode Equivalence

Metamath supports two proof formats: *normal* (explicit label sequence) and *compressed* (a compact encoding). We prove these are equivalent at the database level:

```
theorem
    ProofReachableZ_iff_NormalProofReachable
    (db : DB) (label : String) (fmla : Formula) (stack : Array Formula)
    (h_wf : WellFormedDB db)
    (h_size : stack.size = 1) (h_fmla : stack[0]? = some fmla) :
    ProofReachableZ db label fmla stack
    <-> NormalProofReachable db label fmla stack
```

Listing 5: Mode equivalence (ParserAnyModeEquivalence.lean:236).

This means a reviewer need only understand one proof mode to trust the entire system.

4 The Generic Spec Theorem

Underneath the parser-level biconditionals lies a pure specification-level equivalence between the operational and declarative formulations of provability.

```
theorem operational_iff_semantic
    {G : Database} {consts : ConstSet} {fr : Frame} {e : Expr}
    (h_wf : WellFormedDatabaseStrong G consts)
    (h_fr_nodup : FloatVarNoDup fr)
    (h_fr_disjoint : Spec.FrameVarsDisjointConsts consts fr) :
    Provable G fr e
    <-> Semantic.Provable (dbToAxioms G) (frameToContext fr)
         (exprToFormula (varMapOfFrame fr) e)
```

Listing 6: Spec-level equivalence (`Spec/Equivalence.lean`:3505).

Premises. The theorem requires three structural well-formedness conditions:

- `WellFormedDatabaseStrong`: every assertion in the database has well-formed hypotheses, consistent typecodes, and valid DV pairs.
- `FloatVarNoDup`: floating hypotheses in the frame have distinct variables.
- `FrameVarsDisjointConsts`: frame variables do not collide with database constants.

Discharging from parser success. A specialized bridge theorem (`ParserEquivalence.lean`) discharges all three premises from a single hypothesis: successful `checkBytes` completion (no error). The parser's well-formedness gate (`wellFormed?`) and its structural invariants guarantee that any successfully-parsed database satisfies all three conditions. Concretely, KernelClean's strong well-formedness extractor and ParserEquivalence's total parser bridge package this discharge, so reviewers do not need to re-thread generic premises at call sites. See Listing A.3 in Appendix A.

Bridge functions. The equivalence uses four translation functions:

- `dbToAxioms`: maps the operational `Database` (a partial function from labels to frame–expression pairs) to a predicate on `Statements`.
- `frameToContext`: converts an operational `Frame` to a declarative `Context` (hypothesis list + DV relation).
- `exprToFormula`: converts an operational `Expr` to a declarative `Formula` (typecode–term pair).
- `varMapOfFrame`: extracts the variable-to-typecode mapping from floating hypotheses in the frame.

A reviewer should check that these translations faithfully model the Metamath specification (sections 4.2–4.3 of [1]).

5 Prefix Provenance

5.1 The Circularity Problem

When building a database incrementally (reading a `.mm` file top to bottom), each `$p` theorem is inserted *after* its proof is checked. But can we be sure a theorem doesn't use itself—or a later theorem—in its own proof?

Under the default Metamath configuration, the answer is *no*: the `?` step mechanism (section 4.4.6 of [1]) allows incomplete proofs that push the theorem's own formula onto the stack without any real proof. This is by design—`?` marks a proof as incomplete—but it means the parser cannot guarantee that accepted proofs are genuinely sound in the pre-insertion database.

5.2 The `prefixCertified` Predicate

We define a configuration predicate that is sufficient for prefix provenance:

```
def prefixCertified (c : ModeConfig) : Prop :=
  c.rejectUnknownSteps = true
  and c.allowDuplicateFloat = false
```

Listing 7: `prefixCertified` (`Verify.lean`:132).

Two concrete configurations satisfy this:

- `soundDefault`: minimal strict mode (`rejectUnknownSteps := true`).
- `knife`: stricter mode (also rejects top-level essential hypotheses).

5.3 The Prefix Provenance Theorem

```
theorem checkBytes_done_finishProofEvent_certified
    (arr : ByteArray) (config : ModeConfig)
    (h_cfg : config.prefixCertified)
    (h_success : (checkBytes arr config).error? = none) :
    let s0 : ParserState := { (default : ParserState) with
      db := { (default : DB) with config := config } }
    forall (pos : Nat) (tk : ByteSlice) (pr : ProofState),
      FinishProofEvent (s0.feedAll 0 arr) pos tk pr ->
      exists (G : Spec.Database) (fr : Spec.Frame),
        toDatabase (s0.feedAll 0 arr).db = some G
        and toFrame (s0.feedAll 0 arr).db
              (s0.feedAll 0 arr).db.frame = some fr
        and Spec.Provable G fr (toExpr pr.fmla)
```

Listing 8: Prefix provenance (`PrefixWitnessCheckBytes.lean`:2112).

Reading the theorem. For every `FinishProofEvent` (a moment during parsing when a `$p` proof is completed), the theorem's formula is `Spec.Provable` in the database and frame at the moment of the event—*before* the theorem is inserted. This rules out circular reasoning.

Proof technique: ghost predicates. The proof maintains a `ProofGhost` invariant through the parser loop. This ghost tracks the proof state's stack and heap certificates, the preload phase structure (for compressed proofs), and a scope condition (that the proof frame's hypotheses are a subset of the database frame). The ghost is established when a `$p` token starts a proof and maintained through every `feedToken` call. At `finishProof`, the ghost is discharged to produce `Spec.Provable`.

6 What Is Not Claimed

Honest verification requires documenting not just what is proved but also what is *not* proved.

Include expansion is trusted. The Metamath `$[ ... $]` include mechanism is handled by IO-based pre-processing outside the formalized core. Our theorems cover `checkBytes` given a fully-expanded `ByteArray`.

Error codes: soundness only. The theorem `checkBytes_parseErrorCode?_fullyCertified` (Listing A.4, Appendix A) proves that each of the 55 error codes implies a specific semantic violation. This is *soundness*: "code X implies violation X." We do *not* prove completeness: "if violation X is the first to occur, the parser produces code X." First-violation completeness would require formalizing execution-order priority among checks—tractable but not essential for correctness review.

Mode-indexed vacuous predicates. Some predicates (`TokpInv`, `ProofGhost`, etc.) use `| _ =>` ↪ `True` for modes where the invariant is intentionally vacuous (e.g. `TokpInv` is only constrained in proof mode). These are *not* placeholder theorems or masked `sorry`s—they are deliberate "this mode imposes no constraint" defaults.

`toFrame` returns `Option`. The translation `toFrame : DB -> Frame -> Option Spec.Frame` is genuinely partial: not every implementation `Frame` corresponds to a well-formed spec `Frame`. The main biconditional existentially quantifies over a successful `toFrame` extraction.

No performance claims. Heartbeat budgets and `set_option maxHeartbeats` annotations are engineering accommodations for Lean's kernel, not mathematical claims.

7 Reproducing Mechanically

7.1 Build Instructions

- **Lean toolchain**: `leanprover/lean4:v4.18.0-rc1` (specified in `lean-toolchain`)
- **Dependencies**: Batteries v4.27.0-rc1 (no Mathlib)

```
# Build everything (recommended: set 6GB memory cap)
ulimit -Sv 6291456 && nice -n 19 lake build
# -> Build completed successfully (129 jobs).

# Run test suite
lake build testParserInvariants
```

```
./.lake/build/bin/testParserInvariants
# -> 151/151 tests passed
```

7.2 Verifying Sorry-Freedom

```
grep -r "sorry" Metamath/ --include="*.lean"
# -> (no output: 0 sorries)

grep -r "axiom " Metamath/ --include="*.lean"
# -> only in comments (ParserInvariants.lean before/after example)
```

7.3 Reviewing Strategy

Start at `Metamath/ParserEquivalence.lean`: this is the canonical API surface that re-exports all top-level theorems with a detailed theorem map in its module docstring. From there:

1. Read the theorem *statements* (not proofs) of the main biconditionals.
2. Check the spec definitions (`Spec/Operational.lean`, `DeclarativeSpec.lean`).
3. Check the bridge functions (`toDatabase`, `toFrame`, `toExpr`, `dbToAxioms`, `frameToContext`).
4. Run `lake build` to let Lean's kernel verify every proof.

8 Architecture and Proof Strategy

8.1 Layer Diagram

The verification proceeds bottom-up through five layers:

```
ByteArray
  |
  v
checkBytes (Verify.lean)
  |
  v
WellFormedDB (WellFormedness.lean)
  |
  v
verify_impl_sound / verify_impl_complete (KernelClean.lean)
  |
  v
operational_iff_semantic (Spec/Equivalence.lean)
  |
  v
verify_parser_acceptance_iff_spec_provable (KernelClean.lean)
  |
  v
verify_parser_acceptance_iff_semantic_provable_total
  (ParserEquivalence.lean)
```

8.2 Theorem Chain

The proof proceeds through the following chain (all sorry-free):

1. `checkBytes` parses bytes and validates `wellFormed?` as a final gate.
2. `checkBytes_no_error_wellFormed?`: no error implies `wellFormed? = true`.
3. `wellFormedDB_of_wellFormed?`: `wellFormed? = true` implies `WellFormedDB`.
4. `parser_construction_wellformed`: composes steps 2 and 3.
5. `verify_impl_sound`: the soundness direction (acceptance implies provability).
6. `verify_impl_complete`: the completeness direction (provability implies acceptance).
7. `operational_iff_semantic`: spec-level equivalence between operational and declarative provability.
8. `parser_operational_iff_semantic_total`: specializes the bridge to parser-origin databases with total extraction.
9. The end-to-end biconditionals: combined from steps 5–8.

8.3 Key Proof Techniques

Parser loop induction. The main soundness proof uses induction over the token stream, maintaining a `ParserStateInv` invariant that tracks well-formedness, frame scoping, and hypothesis consistency.

Ghost predicates. For prefix provenance, a `ProofGhost` predicate is maintained through the parser loop. The ghost carries four conjuncts tracking stack/heap certificates, the preload phase structure, and the scope condition for compressed proofs.

Guard combinators. Error code certification uses a `GuardCombinator` framework: each error path is wrapped in a guard that extracts a certified payload and semantic violation evidence in a single pass.

8.4 Statistics

9 Conclusion

We have presented the first complete formal verification of a Metamath proof checker. The central result—an end-to-end biconditional from raw bytes to specification-level provability—establishes that the implementation `checkBytes` is both sound and complete. The biconditional is further bridged to an independent declarative semantics, and under strict configuration, prefix provenance rules out circular reasoning.

Related work. Carneiro's `mm0-rs` [2] verifier is implemented in Rust with a focus on performance; it has not been formally verified. The `mmj2` proof assistant includes extensive testing but no formal correctness proof. The Metamath specification [1] defines the proof system but does not itself come with a machine-checked proof of checker correctness.

Table 2: Project statistics.

Metric	Value
Total Lean lines	54,130
Lean files	46
Build modules	129
Test cases	151
Sorries	0
Axioms	0
Error codes certified	55/55
Largest file	KernelClean.lean (11,395 lines)

Future directions. Three natural extensions remain:

- Formalizing include expansion (`$[ ... $]`), extending the trust boundary to cover file-level composition.
- Error code *completeness*: proving that if a specific violation occurs first, the parser produces the corresponding code.
- Integration with mm0: bridging our Lean formalization to the mm0 proof format used by `mm0-rs`.

A Key Theorem Listings

This appendix reproduces the exact Lean signatures of the main theorems, copied verbatim from the source files. Line numbers refer to the source files at the time of writing (February 2026).

A.1 `Spec.ProofValid` (Operational.lean:49–75)

```
inductive ProofValid (G : Database) :
    Frame -> List Expr -> List ProofStep -> Prop where
  | nil : forall fr, ProofValid G fr [] []
  | useEssential : forall fr stack steps e,
      Hyp.essential e in fr.hyps ->
      ProofValid G fr stack steps ->
      ProofValid G fr (e :: stack)
        (ProofStep.useHyp (Hyp.essential e) :: steps)
  | useFloating : forall fr stack steps c v,
      Hyp.floating c v in fr.hyps ->
      ProofValid G fr stack steps ->
      ProofValid G fr (<c, [v.v]> :: stack)
        (ProofStep.useHyp (Hyp.floating c v) :: steps)
  | useAxiom : forall fr stack steps l fr' e s,
      G l = some (fr', e) ->
      dvOK fr.vars fr'.dv fr.dv s ->
      (forall c v, Hyp.floating c v in fr'.hyps ->
        (s v).typecode = c) ->
      ProofValid G fr stack steps ->
```

```
      forall needed : List Expr,
      needed = fr'.hyps.map (fun h => match h with
        | Hyp.essential e => applySubst fr'.vars s e
        | Hyp.floating _ v => s v) ->
      forall remaining : List Expr,
      stack = needed.reverse ++ remaining ->
      ProofValid G fr (applySubst fr'.vars s e :: remaining)
        (ProofStep.useAssertion l s :: steps)
```

A.2 `operational_iff_semantic` (Equivalence.lean:3505)

```
theorem operational_iff_semantic
    {G : Database} {consts : ConstSet}
    {fr : Frame} {e : Expr}
    (h_wf : WellFormedDatabaseStrong G consts)
    (h_fr_nodup : FloatVarNoDup fr)
    (h_fr_disjoint : Spec.FrameVarsDisjointConsts consts fr) :
    Provable G fr e
    <-> Semantic.Provable (dbToAxioms G) (frameToContext fr)
          (exprToFormula (varMapOfFrame fr) e)
```

A.3 `parser_operational_iff_semantic_total` (ParserEquivalence.lean:380)

```
theorem parser_operational_iff_semantic_total
    (bytes : ByteArray)
    (h_success : (Verify.checkBytes bytes).error? = none)
    (fr : Spec.Frame) (e : Spec.Expr)
    (h_frame : toFrame (Verify.checkBytes bytes)
                (Verify.checkBytes bytes).frame = some fr) :
    Spec.Provable (toDatabaseTotal (Verify.checkBytes bytes)) fr e
    <-> Spec.Semantic.Provable
          (dbToAxioms (toDatabaseTotal (Verify.checkBytes bytes)))
          (frameToContext fr)
          (exprToFormula (varMapOfFrame fr) e)
```

A.4 `checkBytes_parseErrorCode?_fullyCertified` (ErrorCodeSemantics.lean:482)

```
theorem checkBytes_parseErrorCode?_fullyCertified
    (arr : ByteArray) (config : ModeConfig) (code : ParseErrorCode) :
    (checkBytes arr config).parseErrorCode? = some code ->
    CodePayloadWitness (checkBytes arr config) code
    and (checkBytes arr config).RuleSemanticViolation code
```

A.5 `ModeConfig.prefixCertified` and `soundDefault` (Verify.lean)

```
def soundDefault : ModeConfig := {
  rejectUnknownSteps := true
```

```
}

def prefixCertified (c : ModeConfig) : Prop :=
  c.rejectUnknownSteps = true
  and c.allowDuplicateFloat = false

theorem soundDefault_prefixCertified :
    prefixCertified soundDefault := <rfl, rfl>
```

B File Inventory

Table 3 lists all 46 Lean files with line counts.

C Unit Tests and Spec Alignment

This appendix lists unit tests from the `metamath-test` test suite [1], grouped by the Metamath specification rule they exercise. Each test shows the expected verdict (ACCEPT or REJECT) and the complete `.mm` file content. Tests exceeding 70 lines are summarized in a table. The test manifest (`run-testsuite-all`) is the authoritative source; this appendix is auto-generated from it.

C.1 Character Set (L73–79)

> *file are the 94 non-whitespace printable ascii characters, which are digits, upper and lower case letters, and the following 32 special characters: ! " # $ % & ' ( ) * + , - . / : ; ¡ = ¿ ? @ [\ ] ^ _ ‘ { — } ~ plus the following characters which are the "white space" characters: space (a printable character), tab, carriage return, line feed, and form feed. We will use typewriter font to display the printable characters.*

test01_non-printable_ascii_characters.mm [REJECT] *L73-79: only printable ASCII + whitespace allowed*

```
$( Unit Test 1: Non-printable ASCII characters $)
$( Should reject: True $)

$c wff |- $.
$c →$.
$v x $.
wf $f wff x $.
ax $a |- x $.
```

C.2 Whitespace Characters (L77–79)

> *plus the following characters which are the "white space" characters: space (a printable character), tab, carriage return, line feed, and form feed. We will use typewriter font to display the printable characters.*

emptyline.mm [ACCEPT] *L77-79: whitespace (including newlines) is valid separator*

test54_whitespace_variety.mm [ACCEPT] *L77-79: various whitespace chars valid*

```
$( Test 54: Whitespace variety - tabs and spaces are valid whitespace per Sec 4.1.1; should ACCEPT $)

$c  wff |- $.
```

```
$v  ph  $.

f1  $f  wff ph $.
a1  $a  |- ph $.
p1  $p  |- ph $= f1   a1 $.
```

test68_formfeed_whitespace.mm [ACCEPT] *L77-79: form-feed (0x0C) is whitespace*

```
$( Test 68: Form-feed (0x0C) is valid whitespace per Metamath spec section 4.1.1; should ACCEPT $)

$c

wff

|-

$.
$v

ph

$.

f1

$f

wff

ph

$.
a1

$a

|-

ph

$.
p1

$p

|-

ph

$=

f1

a1

$.
```

C.3 Token Separation (L81–83)

separated by white space (which is any sequence of one or more white space characters). The set of keyword tokens is ${, $}, $c, $v, $f, $e, $d, $a, $p, $., $=, $(, $), $[, and $]. The last four are called auxiliary or

comment1.mm [REJECT] *L81-83: whitespace required between tokens*

```
$( $( $)
```

test02_missing_whitespace_between_tokens.mm [REJECT] *L81-83: tokens must be separated by whitespace*

```
$( Unit Test 2: Missing whitespace between tokens $)
$( Should reject: True $)

$cwff$c|-.
$vx$.
wf$fwffx$.
ax$a|-x$.
```

test06_dangling_dollar_sign_at_eof.mm [REJECT] *L81-83: $ must be followed by valid keyword char*

```
$( Unit Test 6: Dangling dollar sign at EOF $)
$( Should reject: True $)

$c wff |- $.
$v x $.
wf $f wff x $.
$
```

test18_missing_whitespace_after_comment.mm [REJECT] *L81-83: whitespace required between tokens*

```
$( Unit Test 18: Missing whitespace after comment $)
$( Should reject: True $)

$c wff $.
$v x $.
$( No space after comment close $)wf $f wff x $.
```

test25_missing_space_before_comment.mm [REJECT] *L81-83: \$(must be whitespace-separated*

```
$( Unit Test 25: Missing space before comment $)
$( Should reject: True $)

$c wff$.$(comment$)
```

test31_missing_whitespace_after_comment.mm [REJECT] *L81-83: \$) must be whitespace-separated*

```
$( Unit Test 31: Missing whitespace after comment $)
$( Should reject: True $)

$( comment $)$c wff $.
```

test49_token_splice_axiom.mm [REJECT] *L81-83: token boundaries must be preserved*

```
$( Unit Test 49: Include as token splice in axiom $)
$( Should reject: True - include inside statement is invalid in strict mode $)
$( Category: CORE - Strict include semantics $)
$( Spec Section 4.1.2: "include directives are processed at outermost level" $)
$( Spec Appendix E: Include not allowed inside statements $)

$c wff |- $.
```

```
$v x $.
fx $f wff x $.

$( Include splices tokens into the middle of this statement - INVALID $)
ax-splice $a $[ ./helpers/test49_helper.mm $]
```

test50_token_splice_proof.mm [REJECT] *L81-83: token boundaries must be preserved*

```
$( Unit Test 50: Include as token splice in proof $)
$( Should reject: True - include inside statement is invalid in strict mode $)
$( Category: CORE - Strict include semantics $)
$( Spec Section 4.1.2: "include directives are processed at outermost level" $)
$( Spec Appendix E: Include not allowed inside statements $)

$c wff |- $.
$v x $.
fx $f wff x $.
ax-x $a |- x $.

$( Include splices proof steps from external file - INVALID $)
th $p |- x $= $[ ./helpers/test50_helper.mm $] $.
```

C.4 Comment Delimiters (L89–91)

4.1. SPECIFICATION OF THE METAMATH LANGUAGE

test03_nested_comment_delimiters.mm [REJECT] *L89-91: \$(may not appear inside comments*

```
$( Unit Test 3: Nested comment delimiters $)
$( Should reject: True $)

$c wff |- $.
$( outer $( nested $) comment $)
$v x $.
wf $f wff x $.
ax $a |- x $.
```

C.5 Outermost Scope (L105–106)

followed by $]. It is only allowed in the outermost scope (i.e., not between ${ and $}) and must not be inside a statement (e.g., it may not occur between

test40_include_scope_correct_outer.mm [REJECT] *L105-106: \$[\$] only allowed in outermost scope*

```
$( Unit Test 40: Include inside block $)
$( Should reject: True - include must be at outermost level $)
$( Category: CORE - Strict include semantics $)
$( Spec Section 4.1.2: Include directives processed at outermost level only $)

$c wff |- $.
$v x $.
wx $f wff x $.
ax-x $a |- x $.

${
  $( Include inside block $)
  $[ ./helpers/test40_helper.mm $]

  $( Use included axiom INSIDE the block - this should work $)
  th1 $p |- y $= wy ax-inner $.
$}
```

```
$( Outside the block, only x is available $)
th2 $p |- x $= wx ax-x $.
```

test47_constant_inner_scope_main.mm [REJECT] *L105-106: \$[\$] only allowed in outermost scope*

```
$( Unit Test 47: Include directive inside block $)
$( Category: CORE - CRITICAL scoping test $)
$( Should reject: True - include must be in outermost scope $)
$( Spec Section 4.1.2 L105-106: "$[ $] only allowed in outermost scope" $)

$c wff |- $.

${
  $( INVALID: Include inside block - must be in outermost scope only $)
  $[ ./helpers/test47_helper.mm $]

  $( Try to use included declarations $)
  th1 $p |- inner-var $= finner ax-inner $.
$}
```

C.6 Block Delimiters (L153–155)

typecode (an active constant), followed by an active variable, followed by the $. token. A $e statement consists of a label, followed by $e, followed by its typecode (an active constant), followed by zero or more active math

test04_unbalanced_block_delimiters.mm [REJECT] *L153-155: \${ must match \$}*

```
$( Unit Test 4: Unbalanced block delimiters $)
$( Should reject: True $)

$c wff |- $.
${
  $v x $.
  wf $f wff x $.
$( missing $} $)
ax $a |- x $.
```

C.7 $f Type Globality (L156–158)

symbols, followed by the $. token. A hypothesis is a $f or $e statement. The type declared by a $f statement for a given label is global even if the variable is not (e.g., a database may not have wff P in one local scope nd

test15_multiple_f_for_same_variable_bad.mm [REJECT] *L156-158: type declared by \$f is GLOBAL (strict)*

```
$( Unit Test 15: Multiple $f for same variable $)
$( Should reject: True $)

$c wff class $.
$v x $.
f1 $f wff x $.
f2 $f class x $.
```

test16_conflicting_typecodes_bad.mm [REJECT] *L156-158: type declared by \$f is GLOBAL (strict)*

```
$( Unit Test 16: Conflicting typecodes $)
$( Should reject: True $)
```

```
$c wff class $.
$v x $.
${
  f1 $f wff x $.
  ${
    f2 $f class x $.
  $}
$}
```

C.8 Constant/Variable Declarations (L161–163)

variables, followed by the $. token. A compound $d statement consists of $d, followed by three or more variables (all different), followed by the $. token. The order of the variables in a $d statement is unimportant. A

local-var-bad.mm [REJECT] *L161-163: variable scope violation*

```
  $c formula $.
  $c A. $.
  $c setvar $.
  $v ph $.
  wph $f formula ph $.

  ${
    $v x $.
    $( This is not valid because there is no $f for the variable ‘ x ‘ $)
    $( Extend formula definition to include the universal quantifier ('for all'). $)
    wal $a formula A. x ph $.
  $}
```

test07_redeclaration_of_constant.mm [REJECT] *L161-163: constant may not be redeclared*

```
$( Unit Test 7: Redeclaration of constant $)
$( Should reject: True $)

$c wff |- $.
$c wff $.
$v x $.
wf $f wff x $.
ax $a |- x $.
```

test45_variable_redeclaration.mm [ACCEPT] *L161-163: variables MAY be redeclared in new scope*

```
$( Unit Test 45: Variable redeclaration across scopes $)
$( Should reject: False - variable can be redeclared after scope ends $)
$( Spec Section 4.2.8: "A variable may be declared again after it becomes inactive" $)

$c wff class |- $.
$v x $.

${
  $( First scope: x has typecode wff $)
  fx1 $f wff x $.
  ax1 $a |- x $.
  $( Use x within its scope $)
  th1 $p |- x $= fx1 ax1 $.
$}

${
  $( Second scope: x has typecode class - VALID redeclaration $)
  fx2 $f class x $.
  ax2 $a class x $.
  $( Use x within its scope $)
  th2 $p class x $= fx2 ax2 $.
```

```
$}
```

test48_variable_conflict_main.mm [REJECT] *L161-163: variable conflicts with constant*

```
$( Unit Test 48: Include causing variable redeclaration conflict $)
$( Category: CORE - CRITICAL scoping test $)
$( Should reject: True - x already active when include tries to redeclare it $)
$( Spec Section 4.2.8: "A variable may not be declared a second time while it is active" $)

$c wff |- $.
$v x $.
fx $f wff x $.

$( Include file that also declares $v x - should fail $)
$[ ./helpers/test48_helper.mm $]
```

C.9 Label Syntax (L167–169)

A $d statement is also called a disjoint (or distinct) variable restriction. A $a statement consists of a label, followed by $a, followed by its typecode (an active constant), followed by zero or more active math symbols,

demo0-illegal-label-bad1.mm [REJECT] *L167-169: labels must be alphanumeric + hyphen + underscore + period*

```
$( demo0.mm 1-Jan-04 $)

$(
                    PUBLIC DOMAIN DEDICATION

This file is placed in the public domain per the Creative Commons Public
Domain Dedication. ht\tp://creativecommons.org/licenses/publicdomain/

Norman Megill - email: nm at alum.mit.edu

$)


$( This file is the introductory formal system example described
   in Chapter 2 of the Metamath book. $)

$( Declare the constant symbols we will use $)
    $c 0 + = -> ( ) term wff |- $.
$( Declare the metavariables we will use $)
    $v t r s P Q $.
$( Specify properties of the metavariables $)
    t\t $f term t $.
    tr $f term r $.
    ts $f term s $.
    wp $f wff P $.
    wq $f wff Q $.
$( Define "term" (part 1) $)
    (tze $a term 0 $.
$( Define "term" (part 2) $)
    t)pl $a term ( t + r ) $.
$( Define "wff" (part 1) $)
    w"eq $a wff t = r $.
$( Define "wff" (part 2) $)
    w%i\m $a wff ( P -> Q ) $.
$( State axiom a1 $)
    a1 $a |- ( t = r -> ( t = s -> r = s ) ) $.
$( State axiom 'a2' $)
    'a2' $a |- ( t + 0 ) = t $.
    ${
       min $e |- P $.
```

```
      maj $e |- ( P -> Q ) $.
$( Define the modus ponens inference rule $)
      mp  $a |- Q $.
   $}
$( Prove a theorem $)
   th1 $p |- t = t $=
  $( Here is its proof: $)
      t\t (tze t)pl t\t w"eq t\t t\t w"eq t\t 'a2' t\t (tze t)pl
      t\t w"eq t\t (tze t)pl t\t w"eq t\t t\t w"eq w%i\m t\t 'a2'
      t\t (tze t)pl t\t t\t a1 mp mp
    $.
```

underscores.mm [ACCEPT] *L167-169: underscores allowed in labels*

```
  $c a $.

  $( This is _italic_ and the command "MM-PA> MINIMIZE__WITH *" and a
     sub_script, a label ~ lab[[''~~__ , and a url: ~ http://a_b__c.com/[[''~~__.
     Doubled chars: [[ '' ~~ __
     <HTML>
     Inside HTML tags:

     This is _italic_ and the command "MM-PA> MINIMIZE__WITH *" and a
     sub_script, a label ~ lab[[''~~__ , and a url: ~ http://a_b__c.com[[''~~__.
     Doubled chars: [[ '' ~~ __
     </HTML> $)
  underscores $a a $.

$( $t
 htmldef "a" as "a";
  althtmldef "a" as "a";
   latexdef "a" as "a"; $)
```

test57_case_sensitivity_labels.mm [ACCEPT] *L167-169: labels are case-sensitive*

```
$( Test 57: Case sensitivity for labels - labels are case-sensitive per Sec 4.1.3; should ACCEPT $)

$c wff |- $.
$v ph $.

f1 $f wff ph $.
ax $a |- ph $.
Ax $a |- ph $.

p1 $p |- ph $= f1 ax $.
p2 $p |- ph $= f1 Ax $.
```

C.10 $f Scope (L174–175)

A $f, $e, or $d statement is active from the place it occurs until the end of the block it occurs in. A $a or $p statement is active from the place

test35_f_not_active_after_block_close.mm [REJECT] *L174-175: \$f active only until block end*

```
$( Unit Test 35: $f not active after block close $)
$( Should reject: True $)

$c wff |- $.
${
  $v x $.
  wf $f wff x $.
$}
$( x and wf are no longer active $)
bad $a |- x $.
```

C.11 Variable Scope (L174–178)

A $f, $e, or $d statement is active from the place it occurs until the end of the block it occurs in. A $a or $p statement is active from the place it occurs through the end of the database. There may not be two active $f statements containing the same variable. Each variable in a $e, $a, or $p statement must exist in an active $f statement.3

test08_variable_used_outside_scope.mm [REJECT] *L174-178: variable must have active \$f*

```
$( Unit Test 8: Variable used outside scope $)
$( Should reject: True $)

$c wff |- $.
${
  $v x $.
  wf $f wff x $.
$}
bad $a |- x $.
```

test13_f_with_undeclared_variable.mm [REJECT] *L174-178: \$f variable must be declared*

```
$( Unit Test 13: $f with undeclared variable $)
$( Should reject: True $)

$c wff |- $.
bad $f wff undeclared $.
```

C.12 Variable Must Have $f (L177–178)

statements containing the same variable. Each variable in a $e, $a, or $p statement must exist in an active $f statement.3

test14_variable_without_f_hypothesis.mm [REJECT] *L177-178: each variable in \$e/\$a/\$p must have active \$f*

```
$( Unit Test 14: Variable without $f hypothesis $)
$( Should reject: True $)

$c wff |- $.
$v x $.
bad $a |- x $.
```

C.13 Label Uniqueness (L179–180)

Each label token must be unique, and no label token may match any math symbol token.4

test10_duplicate_labels.mm [REJECT] *L179-180: label tokens must be unique*

```
$( Unit Test 10: Duplicate labels $)
$( Should reject: True $)

$c wff |- $.
$v x $.
dup $f wff x $.
dup $a |- x $.
```

test11_label_conflicts_with_symbol.mm [REJECT] *L179-180: no label may match any math symbol*

```
$( Unit Test 11: Label conflicts with symbol $)
$( Should reject: True $)
```

```
$c wff |- $.
$v x $.
wff $f wff x $.
```

test34_label_token_in_math_context.mm [REJECT] *L179-180,339: labels and math symbols are disjoint*

```
$( Unit Test 34: Label token in math context $)
$( Should reject: True $)

$c wff |- $.
$v x $.
wf $f wff x $.
$( Using 'wf' where math symbol expected $)
bad $a wf x $.
```

C.14 Mandatory Hypotheses (L182–184)

> *of (zero or more) variables in the assertion and in any active $e statements. The (possibly empty) set of mandatory hypotheses is the set of all active $f statements containing mandatory variables, together with all active $e*

test64_multiple_ehyp_order.mm [ACCEPT] *L182-184, L211-214: mandatory hyp ordering*

```
$( test64_multiple_ehyp_order.mm - ACCEPT
   Tests spec 4.1.4 L182-184, L211-214:
   "The (possibly empty) set of mandatory hypotheses is the set of all
   active $f statements containing mandatory variables, together with
   all active $e statements."

   "causes them to match the topmost entries of the stack, in order of
   occurrence of the mandatory hypotheses"

   Key insight: mandatory hypotheses = $f (for mandatory vars) + $e (all active)
   in order of OCCURRENCE in the file.
$)

$c wff |- ( ) -> $.
$v p q $.
wp $f wff p $.
wq $f wff q $.

$( Build implication $)
wi $a wff ( p -> q ) $.

$( A theorem with two essential hypotheses.
   Mandatory hyps in occurrence order: wp, wq, e1, e2 $)
${
  e1 $e |- p $.
  e2 $e |- ( p -> q ) $.
  $( Modus ponens: from p and p->q, derive q $)
  mp $a |- q $.
$}

$( Use mp with concrete values.
   To apply mp where p:=( p -> p ), q:=( p -> p ):
   Push order: wp (for p), wp (for q), inst of e1, inst of e2

   We'll prove: from ( p -> p ) and ( ( p -> p ) -> ( p -> p ) ), get ( p -> p )
$)

${
  h1 $e |- ( p -> p ) $.
  h2 $e |- ( ( p -> p ) -> ( p -> p ) ) $.
```

```
  $( Stack for mp: wp wp wi, wp wp wi, h1, h2
     This substitutes p:=( p -> p ), q:=( p -> p ) $)
  result $p |- ( p -> p ) $=
    wp wp wi wp wp wi h1 h2 mp $.
$}
```

C.15 Self-Substitution (L204–205)

An expression is a sequence of math symbols. A substitution map associates a set of variables with a set of expressions. It is acceptable for a

test62_self_substitution.mm [ACCEPT] *L204-205: variable mapped to expression containing itself*

```
$( test62_self_substitution.mm - ACCEPT
   Tests spec 4.1.4 L204-205:
   "It is acceptable for a variable to be mapped to an expression
   containing it."

   This tests that p can be substituted with an expression containing p.
$)

$c wff |- ( ) -> $.
$v p q $.
wp $f wff p $.
wq $f wff q $.

$( Build implication: wff ( p -> q ) $)
wi $a wff ( p -> q ) $.

$( Axiom: |- ( p -> p )
   Mandatory hypothesis: just wp (only var p is used) $)
ax-id $a |- ( p -> p ) $.

$( Prove ( ( p -> p ) -> ( p -> p ) )
   We apply ax-id with p := ( p -> p )
   This means p maps to an expression containing p itself!

   ax-id needs: wp
   We need to push wff ( p -> p ) to substitute for p.
   Building wff ( p -> p ): wp wp wi (gives wff ( p -> p ))
   Then apply ax-id.
$)
th-self $p |- ( ( p -> p ) -> ( p -> p ) ) $=
  wp wp wi ax-id $.
```

C.16 Typecode Matching (L208–212)

the expressions that the variables map to. A proof is scanned in order of its label sequence. If the label refers to an active hypothesis, the expression in the hypothesis is pushed onto a stack. If the label refers to an assertion, a (unique) substitution must exist that, when made to the mandatory hypotheses of the referenced assertion, causes

test22_typecode_mismatch_in_substitution.mm [REJECT] *L208-212: substitution must match typecode*

```
$( Unit Test 22: Typecode mismatch in substitution $)
$( Should reject: True $)

$c wff class |- $.
$v x y $.
f1 $f wff x $.
```

```
f2 $f class y $.
ax $a |- x $.
bad $p |- y $= f2 ax $.
```

C.17 Proof Labels (L213–218)

them to match the topmost (i.e. most recent) entries of the stack, in order of occurrence of the mandatory hypotheses, with the topmost stack entry matching the last mandatory hypothesis of the referenced assertion. As many stack entries as there are mandatory hypotheses are then popped from the stack. The same substitution is made to the referenced assertion, and the result is pushed onto the stack. After the last label in the proof is processed,

test21_self-referential_proof.mm [REJECT] *L213-218: proof may not reference itself*

```
$( Unit Test 21: Self-referential proof $)
$( Should reject: True $)

$c wff |- -> ( ) $.
$v x $.
wf $f wff x $.
evil $p |- ( x -> x ) $= ( evil ) A $.
```

test24_undefined_label_in_proof.mm [REJECT] *L213-218: proof labels must be defined*

```
$( Unit Test 24: Undefined label in proof $)
$( Should reject: True $)

$c wff |- $.
$v x $.
wf $f wff x $.
ax $a |- x $.
bad $p |- x $= nosuchlabel $.
```

test32_forward_reference_in_proof.mm [REJECT] *L213-218: proof labels must be previously defined*

```
$( Unit Test 32: Forward reference in proof $)
$( Should reject: True $)

$c wff |- $.
$v x $.
wf $f wff x $.
early $p |- x $= wf later $.
later $a |- x $.
```

C.18 Stack Matching (L218–220)

result is pushed onto the stack. After the last label in the proof is processed, the stack must have a single entry that matches the expression in the $p statement containing the proof.

test26_wrong_conclusion_in_proof.mm [REJECT] *L218-220: final stack must match \$p statement*

```
$( Unit Test 26: Wrong conclusion in proof $)
$( Should reject: True $)

$c wff |- $.
$v x y $.
f1 $f wff x $.
f2 $f wff y $.
ax $a |- x $.
bad $p |- y $= f1 ax $.
```

test33_compressed_proof_stack_underflow.mm [REJECT] *L218-220: stack must not underflow*

```
$( Unit Test 33: Compressed proof stack underflow $)
$( Should reject: True $)

$c wff |- $.
$v x $.
wf $f wff x $.
ax $a |- x $.
$( C tries to pop 3rd item, but stack has only 1 $)
bad $p |- x $= ( ax ) C $.
```

C.19 Unknown Steps (L221–223)

A proof may contain a ? in place of a label to indicate an unknown step (Section 4.4.6). A proof verifier may ignore any proof containing ? but should warn the user that the proof is incomplete.

C.20 Typecodes in $f (L286–292)

Constant symbol declaration Variable symbol declaration Disjoint variable restriction Variable-type ("floating") hypothesis Logical ("essential") hypothesis Axiomatic assertion Provable assertion

test12_non-constant_typecode.mm [REJECT] *L286-292: first symbol in \$f must be constant (typecode)*

```
$( Unit Test 12: Non-constant typecode $)
$( Should reject: True $)

$c wff |- $.
$v x y $.
bad $f x y $.
```

C.21 Math Symbol Syntax (L339–342)

Math symbols are tokens used to represent the symbols that appear in ordinary mathematical formulas. They may consist of any combination of the 93 non-whitespace printable ascii characters other than $. Some examples are x, +, (, —-, !%@?&, and bounded. For readability, it is best

demo0-dollar-in-math-symbol-bad1.mm [REJECT] *L339-342: math symbols must not contain $*

```
$( demo0.mm 1-Jan-04 $)

$(
                    PUBLIC DOMAIN DEDICATION

This file is placed in the public domain per the Creative Commons Public
Domain Dedication. http://creativecommons.org/licenses/publicdomain/

Norman Megill - email: nm at alum.mit.edu

$)


$( This file is the introductory formal system example described
   in Chapter 2 of the Meamath book. $)

$( Declare the constant symbols we will use $)
    $c 0 + = -> ( ) meep$ term wff |- $.
$( Declare the metavariables we will use $)
    $v t r s P Q $.
```

test05_dollar_sign_in_math_symbols.mm [REJECT] *L339-342: math symbols must not contain $*

```
$( Unit Test 5: Dollar sign in math symbols $)
$( Should reject: True $)

$c wff |- $.
$c a$b $.
$v x $.
wf $f wff x $.
ax $a |- x $.
```

test58_case_sensitivity_symbols.mm [ACCEPT] *L339-342: math symbols are case-sensitive*

```
$( Test 58: Case sensitivity for symbols - constants are case-sensitive per Sec 4.1.3; should ACCEPT $)

$c WFF PH |- $.
$v ph $.

f1 $f WFF ph $.
a1 $a |- ph $.
p1 $p |- ph $= f1 a1 $.
```

C.22 $d Syntax (L547–549)

for each possible pair of variables in the original $d statement. For example, $d w x y z $. is equivalent to

test09_d_with_non-variables.mm [REJECT] *L547-549: \$d must contain only variables*

```
$( Unit Test 9: $d with non-variables $)
$( Should reject: True $)

$c wff |- $.
$d wff |- $.
```

C.23 Duplicate $d (L553–558)

$d x y $. $d x z $. $d y z $. Two or more simple $d statements specifying the same variable pair are internally combined into a single $d statement. Thus the set of three statements

test56_duplicate_d_normalization_bad.mm [REJECT] *L553-558: duplicate \$d is error*

```
$( Test 56: Duplicate $d normalization - duplicates allowed; should ACCEPT per Sec 4.2.4 (union of DV pairs)
  ↪ $)

$c wff |- $.
$v x $.

$d x x $.
$d x x $.

f1 $f wff x $.
a1 $a |- x $.
p1 $p |- x $= f1 a1 $.
```

C.24 $c Outermost Scope (L1088)

All $c statements must be placed in the outermost block. Since they are

test47b_constant_inner_scope_direct.mm [REJECT] *L1088: \$c must be in outermost block*

```
$( Unit Test 47b: $c declaration directly in inner scope $)
$( Category: CORE - CRITICAL scoping test $)
$( Should reject: True - $c must be in outermost block $)
$( Spec Section 4.1.2 L1088: "All $c statements must be placed in the outermost block" $)

$c wff |- $.

${
  $( INVALID: $c declaration inside block - must be in outermost scope only $)
  $c inner-const $.

  $v x $.
  wx $f wff x $.
  ax-x $a |- x $.
$}
```

C.25 Missing $p (L1114–1115)

A constant may not be redeclared. And, as mentioned above, constants must be declared in the outermost block.

missing-dollar-p.mm [REJECT] *L1114-1115: missing \$p statement*

```
${ mytoken $}
```

C.26 Compressed Proof ? (L1894–1895)

if desired. In this case, you will see one or more ?'s in the compressed-proof part.

test30_qmark_in_compressed_proof.mm [ACCEPT] *L1894-1895: ? allowed in compressed proofs*

```
$( Unit Test 30: ? in compressed proof $)
$( Should reject: False $)

$c wff |- $.
$v x $.
wf $f wff x $.
ax $a |- x $.
incomplete $p |- x $= ( ax ) ? $.
```

C.27 Proof Verification Failures (§4.2.1)

anatomy-bad1.mm [REJECT] *4.2.1: proof verification failure*

```
$( anatomy.mm $)

$(
                    PUBLIC DOMAIN DEDICATION

This file is placed in the public domain per the Creative Commons Public
Domain Dedication. http://creativecommons.org/licenses/publicdomain/

Norman Megill - email: nm at alum.mit.edu

$)


$( This file is the introductory formal system example described
   in Chapter 4 of the Metamath book, "Anatomy of a proof". $)
```

```
$( Declare the constant symbols we will use $)
$c ( ) -> wff $.
$( Declare the metavariables we will use $)
$v p q r s $.
$( Declare wffs $)
wp $f wff p $.
wq $f wff q $.
wr $f wff r $.
ws $f wff s $.
w2 $a wff ( p -> q ) $.

$( Prove a simple theorem. $)
wnew $p wff ( s -> ( r -> p ) ) $= ws wr wp w2 ws $.
```

anatomy-bad2.mm [REJECT] *4.2.1: proof verification failure*

```
$( anatomy.mm $)

$(
                    PUBLIC DOMAIN DEDICATION

This file is placed in the public domain per the Creative Commons Public
Domain Dedication. http://creativecommons.org/licenses/publicdomain/

Norman Megill - email: nm at alum.mit.edu

$)


$( This file is the introductory formal system example described
   in Chapter 4 of the Metamath book, "Anatomy of a proof". $)

$( Declare the constant symbols we will use $)
$c ( ) -> wff $.
$( Declare the metavariables we will use $)
$v p q r s $.
$( Declare wffs $)
wp $f wff p $.
wq $f wff q $.
wr $f wff r $.
ws $f wff s $.
w2 $a wff ( p -> q ) $.

$( Prove a simple theorem. $)
wnew $p wff ( s -> ( r -> p ) ) $= ws wr wp w2 $.
```

anatomy-bad3.mm [REJECT] *4.2.1: proof verification failure*

```
$( anatomy.mm $)

$(
                    PUBLIC DOMAIN DEDICATION

This file is placed in the public domain per the Creative Commons Public
Domain Dedication. http://creativecommons.org/licenses/publicdomain/

Norman Megill - email: nm at alum.mit.edu

$)


$( This file is the introductory formal system example described
   in Chapter 4 of the Metamath book, "Anatomy of a proof". $)

$( Declare the constant symbols we will use $)
$c ( ) -> wff $.
$( Declare the metavariables we will use $)
$v p q r s $.
```

```
$( Declare wffs $)
wp $f wff p $.
wq $f wff q $.
wr $f wff r $.
ws $f wff s $.
w2 $a wff ( p -> q ) $.

$( Prove a simple theorem. $)
wnew $p wff ( s -> ( r -> p ) ) $= wr wp w2 w2 $.
```

demo0-bad1.mm [REJECT] *4.2.1: proof verification failure*

```
$( demo0.mm 1-Jan-04 $)

$(
                    PUBLIC DOMAIN DEDICATION

This file is placed in the public domain per the Creative Commons Public
Domain Dedication. http://creativecommons.org/licenses/publicdomain/

Norman Megill - email: nm at alum.mit.edu

$)


$( This file is the introductory formal system example described
   in Chapter 2 of the Metamath book. $)

$( Declare the constant symbols we will use $)
    $c 0 + = -> ( ) term wff |- $.
$( Declare the metavariables we will use $)
    $v t r s P Q $.
$( Specify properties of the metavariables $)
    tt $f term t $.
    tr $f term r $.
    ts $f term s $.
    wp $f wff P $.
    wq $f wff Q $.
$( Define "term" (part 1) $)
    tze $a term 0 $.
$( Define "term" (part 2) $)
    tpl $a term ( t + r ) $.
$( Define "wff" (part 1) $)
    weq $a wff t = r $.
$( Define "wff" (part 2) $)
    wim $a wff ( P -> Q ) $.
$( State axiom a1 $)
    a1 $a |- ( t = r -> ( t = s -> r = s ) ) $.
$( State axiom a2 $)
    a2 $a |- ( t + 0 ) = t $.
    ${
       min $e |- P $.
       maj $e |- ( P -> Q ) $.
$( Define the modus ponens inference rule $)
       mp  $a |- Q $.
    $}
$( Prove a theorem $)
    th1 $p |- t = t $=
  $( Here is its proof: $)
       tt tze tt tpl weq tt tt weq tt a2 tt tze tpl
       tt weq tt tze tpl tt weq tt tt weq wim tt a2
       tt tze tpl tt tt a1 mp mp
     $.
```

issue107.mm [REJECT] *4.2.1: proof verification failure (missing hypothesis)*

```
$c axiom hypothesis thesis $.
```

```
ax $a axiom $.

${
   hyp $e hypothesis $.
   mp $a thesis $.
$}

th $p thesis $= ax mp $.
```

C.28 Active Hypotheses (§4.2.5)

test23_using_non-essential_hypotheses.mm [REJECT] *4.2.5: only active hypotheses may be used*

```
$( Unit Test 23: Using non-essential hypotheses $)
$( Should reject: True $)

$c wff |- $.
$v x $.
wf $f wff x $.
${
  hyp $e |- x $.
  th1 $p |- x $= hyp $.
$}
evil $p |- x $= ( hyp ) A $.
```

C.29 $d Symbol Requirements (§4.2.6)

test69_djvars_error_masking.mm [REJECT] *4.2.6: \$d symbols must be declared variables*

```
$( Test 69: $d with undeclared symbol should reject. $)
$( This currently exposes a parser diagnostic masking path in mm-lean4:
   the immediate token error can be overwritten by EOF unclosed-$d reporting. $)

$c wff $.
$v x $.

$d x y $.
```

C.30 Disjoint Variable Violations (§4.2.7)

disjoint1.mm [REJECT] *4.2.7: \$d violation in proof*

```
$c var formula |- $.
$v x y $.
xf $f formula x $.
yf $f formula y $.
${
  $d x y $.
  ax-1 $a |- x y $.
$}
bad $p |- x x $=
  xf xf ax-1 $.
```

disjoint2.mm [REJECT] *4.2.7: \$d violation in proof*

```
$c formula |- ( ) $.

$v x y z w $.
xf $f formula x $.
yf $f formula y $.
zf $f formula z $.
wf $f formula w $.
```

```
combo $a formula ( x y ) $.
${
  $d x y $.
  ax-1 $a |- ( x y ) $.
$}

verybad $p |- ( ( x y ) ( z x ) ) $=
  xf yf combo zf xf combo ax-1 $.

${
  $d x y z $.
  stillbad $p |- ( ( x y ) ( z x ) ) $=
    xf yf combo zf xf combo ax-1 $.
$}

semigood $p |- ( ( x y ) ( z w ) ) $=
  xf yf combo zf wf combo ax-1 $.

${
  $d x y $.
  $d z w $.
  almostgood $p |- ( ( x y ) ( z w ) ) $=
    xf yf combo zf wf combo ax-1 $.
$}

${
  $d x y z w $.
  good $p |- ( ( x y ) ( z w ) ) $=
    xf yf combo zf wf combo ax-1 $.
$}

${
  $d x z $. $d x w $. $d y z $. $d y w $.
  stillgood $p |- ( ( x y ) ( z w ) ) $=
    xf yf combo zf wf combo ax-1 $.
$}
```

disjoint3.mm [REJECT] *4.2.7: \$d violation in proof*

```
$c var formula |- $.
$v x y $.
xf $f formula x $.
yf $f formula y $.
${
  $d x y $.
  ax-1 $a |- x y $.
$}
bad $p |- x y $=
  xf yf ax-1 $.
```

test27_disjoint_variable_constraint_violation.mm [REJECT] *4.2.7: \$d constraints must be satisfied*

```
$( Unit Test 27: Disjoint variable constraint violation $)
$( Should reject: True $)

$c wff -> |- ( ) $.
$v x y z $.
wfx $f wff x $.
wfy $f wff y $.
wfz $f wff z $.
$d x y $.

$( Axiom mentions BOTH x and y, so $d x y is mandatory for it $)
axxy $a |- ( x -> y ) $.

$( This proof violates $d: substitutes z for both x and y $)
$( x:=z, y:=z violates $d x y $)
```

```
bad $p |- ( z -> z ) $= wfz wfz axxy $.
```

C.31 Include Processing (§4.2.8)

test17_include_scope_violation.mm [REJECT] *4.2.8: \$[\$] scope rules*

```
$( Unit Test 17: Include inside block with scope violation $)
$( Should reject: True $)

$c wff |- $.
$v x $.
wx $f wff x $.

${
  $( Include inside block - contents scoped to block $)
  $[ ./helpers/test17_helper.mm $]

  $( This would work - using inside the block $)
  $( th1 $p |- y $= wy ax-inner $. $)
$}

$( Try to use ax-inner outside the block - SCOPE VIOLATION $)
th2 $p |- y $= wy ax-inner $.
```

test28_self_include.mm [REJECT] *4.2.8: file may not include itself*

```
$( Unit Test 28: Self-include $)
$( Should reject: False - Spec section 4.1.2 says "will simply be ignored" $)
$( Note: metamath.exe REJECTS (spec divergence), mmverify_pure.py ACCEPTS (spec-compliant) $)

$c wff $.

$( Include this file itself - causes duplicate declarations $)
$[ ./test28_self_include.mm $]
```

test44_include_cycle_main.mm [REJECT] *4.2.8: circular include detected*

```
$( Unit Test 44: Cycle detection - A includes B, B includes A $)
$( Should reject: False - cycle should be detected and ignored $)
$( Spec: Include stack tracking prevents infinite loops $)

$c wff |- $.

$( Include A, which includes B, which tries to include A again $)
$[ ./helpers/test44_helper_a.mm $]

$( Both A and B should be included exactly once $)
$( Use declarations from both files $)
th-a $p |- var-a $= fa-cycle ax-a-cycle $.
th-b $p |- var-b $= fb-cycle ax-b-cycle $.
```

test46_duplicate_include_main.mm [REJECT] *4.2.8: duplicate include processing*

```
$( Unit Test 46: Duplicate include in different inner blocks $)
$( Should reject: True - includes must be at outermost level $)
$( Category: CORE - Strict include semantics $)
$( Spec Section 4.1.2: Include directives processed at outermost level only $)

$c wff |- $.

${
  $( First include: processes the file $)
  $[ ./helpers/test46_helper.mm $]

  $( Use the included content $)
  th1 $p |- y $= wy ax-y $.
$}
```

```
$${
  $( Second include: should be ignored as whitespace $)
  $[ ./helpers/test46_helper.mm $]

  $( This would fail if file processed twice (y redeclared) $)
  $( But since second include ignored, ax-y and wy are still available globally $)
  th2 $p |- y $= wy ax-y $.
$}
```

test52_include_inside_e.mm [REJECT] *4.2.8: \$[not allowed inside \$e*

```
$( Test 52: $[ ... $] inside an $e statement - must REJECT per Sec 4.1.2 (includes only at outermost level) $
  ↪ )

$c wff |- $.
$v ph $.

e1 $e $[ ./helpers/test40_include_scope_correct_inner.mm $] $.
```

test53_include_inside_d.mm [REJECT] *4.2.8: \$[not allowed inside \$d*

```
$( Test 53: $[ ... $] inside a $d statement - must REJECT per Sec 4.1.2 (includes only at outermost level)
  ↪ and Sec 4.2.4 (varlist only) $)

$c wff |- $.
$v x y $.

$d $[ ./helpers/test40_include_scope_correct_inner.mm $] x y $.
```

C.32 Compressed Proof Format (§4.4)

out-of-range-saved-step-bad1.mm [REJECT] *4.4: compressed proof Z reference out of range*

```
$( out-of-range-saved-step-bad1.mm
   Incorrect compressed proof: An example generated by ChatGPT o3
   That references a proof index greater than the number of saved proof steps.
   Added because mmverify.py was missing a raise MMError and tests did not flag it. $)

$c |- T $.

ax    $a |- T $.
buggy $p |- T $= ( ax ) A Z C $.
```

C.33 Compressed Proof Encoding (Appendix B)

test19_illegal_characters_in_compressed_proof.mm [REJECT] *AppB: compressed proof uses only A-Z and ? (strict)*

```
$( Unit Test 19: Illegal characters in compressed proof $)
$( Should reject: True $)

$c wff |- $.
$v x $.
wf $f wff x $.
ax $a |- x $.
th $p |- x $= ( ax ) A0B $.
```

test29_compressed_proof_header_mismatch.mm [REJECT] *AppB: compressed proof header must match mandatory hyps*

```
$( Unit Test 29: Compressed proof header mismatch $)
$( Should reject: True $)

$c wff |- $.
```

```
$v x $.
wf $f wff x $.
ax1 $a |- x $.
ax2 $a |- x $.
$( Header lists ax1 but proof uses ax2! $)
bad $p |- x $= ( ax1 ) B $.
```

test36_whitespace_in_compressed_proof_bad.mm [REJECT] *AppB: no whitespace inside compressed proof string*

```
$( Unit Test 36: Whitespace in compressed proof $)
$( Should reject: True - This database has invalid proof steps B and C $)
$( Original intent was to test whitespace handling, but ABC is invalid $)

$c wff |- $.
$v x $.
wf $f wff x $.
ax $a |- x $.
$( Compressed proof ABC: A=0 (wf), B=1 (ax), C=2 (out of bounds!) $)
th $p |- x $= ( ax ) A
  B
  C $.
```

test38_whitespace_in_valid_compressed.mm [ACCEPT] *AppB: whitespace allowed in header, not proof string*

```
$( Unit Test 38: Whitespace in valid compressed proof $)
$( Should reject: False - whitespace should be ignored per spec section 4.4.2 $)

$c wff |- $.
$v x $.
wf $f wff x $.
ax $a |- x $.

$( Compressed proof with whitespace: spaces, newlines, tabs between valid steps $)
$( Proof is just: push wf (A), then ax (B) $)
$( Label list: [wf=0, ax=1], so A B is valid $)
th $p |- x $= ( ax ) A
  B   $.
```

test55_compressed_z_oob.mm [REJECT] *AppB: Z reference out of bounds*

```
$( Test 55: Compressed proof index out of bounds - must REJECT per Appendix B (label index must be within
  ↪ header list) $)

$c wff |- $.
$v ph $.
f1 $f wff ph $.
a1 $a |- ph $.

p1 $p |- ph $= ( a1 ) B $.
```

test56_compressed_z_simple_bad.mm [REJECT] *AppB: compressed Z out of bounds*

```
$( Minimal test of compressed proof with Z (save) marker $)

$c wff |- $.
$v ph ps $.
wph $f wff ph $.
wps $f wff ps $.

$( A simple axiom - implication introduction $)
ax-1 $a |- ( ph -> ( ps -> ph ) ) $.

$( Axiom using ax-1 as basis $)
wi $a wff ( ph -> ps ) $.

$( Prove something using Z - save and reuse $)
```

```
$( This proof:
   - A = index 0 = wph (mandatory f-hyp for ph)
   - B = index 1 = wps (mandatory f-hyp for ps)
   - C = index 2 = wi (explicit label)
   - D = index 3 = ax-1 (explicit label)
   So: ( wi ax-1 ) A B C Z D means:
   1. Push wph -> get "wff ph" on stack
   2. Push wps -> get "wff ps" on stack
   3. Apply wi -> get "wff ( ph -> ps )" on stack
   4. Z = save top (which is "wff ( ph -> ps )")
   5. Apply ax-1 with substitutions -> get "|- ( ph -> ( ps -> ph ) )"
   Actually, let me simplify...
$)

$( Simple proof without Z first to verify basic compressed proofs work $)
simple1 $p wff ( ph -> ps ) $= ( wi ) AB $.

$( Now with Z $)
$.
```

C.34 Valid Databases (Positive Tests)

anatomy.mm [ACCEPT] *Valid database per spec*

```
$( anatomy.mm $)

$(
                    PUBLIC DOMAIN DEDICATION

This file is placed in the public domain per the Creative Commons Public
Domain Dedication. http://creativecommons.org/licenses/publicdomain/

Norman Megill - email: nm at alum.mit.edu

$)


$( This file is the introductory formal system example described
   in Chapter 4 of the Metamath book, "Anatomy of a proof". $)

$( Declare the constant symbols we will use $)
$c ( ) -> wff $.
$( Declare the metavariables we will use $)
$v p q r s $.
$( Declare wffs $)
wp $f wff p $.
wq $f wff q $.
wr $f wff r $.
ws $f wff s $.
w2 $a wff ( p -> q ) $.

$( Prove a simple theorem. $)
wnew $p wff ( s -> ( r -> p ) ) $= ws wr wp w2 w2 $.
```

demo0-includee.mm [ACCEPT] *Valid database per spec*

```
$( demo0-includee.mm 1-Jan-04 $)

$(
                    PUBLIC DOMAIN DEDICATION

This file is placed in the public domain per the Creative Commons Public
Domain Dedication. http://creativecommons.org/licenses/publicdomain/

Norman Megill - email: nm at alum.mit.edu

$)
```

```
$( This file is the introductory formal system example described
   in Chapter 2 of the Meamath book. $)

$( Declare the constant symbols we will use $)
    $c 0 + = -> ( ) term wff |- $.
$( Declare the metavariables we will use $)
    $v t r s P Q $.
$( Specify properties of the metavariables $)
    tt $f term t $.
    tr $f term r $.
    ts $f term s $.
    wp $f wff P $.
    wq $f wff Q $.
$( Define "term" (part 1) $)
    tze $a term 0 $.
$( Define "term" (part 2) $)
    tpl $a term ( t + r ) $.
$( Define "wff" (part 1) $)
    weq $a wff t = r $.
$( Define "wff" (part 2) $)
    wim $a wff ( P -> Q ) $.
$( State axiom a1 $)
    a1 $a |- ( t = r -> ( t = s -> r = s ) ) $.
$( State axiom a2 $)
    a2 $a |- ( t + 0 ) = t $.
    ${
       min $e |- P $.
       maj $e |- ( P -> Q ) $.
$( Define the modus ponens inference rule $)
       mp  $a |- Q $.
    $}
```

demo0-includer.mm [ACCEPT] *Valid database per spec*

```
$( demo0-includer.mm 1-Jan-04 $)

$(
                    PUBLIC DOMAIN DEDICATION

This file is placed in the public domain per the Creative Commons Public
Domain Dedication. http://creativecommons.org/licenses/publicdomain/

Norman Megill - email: nm at alum.mit.edu

$)


$[ demo0-includee.mm $]

$( Prove a theorem $)
    th1 $p |- t = t $=
  $( Here is its proof: $)
       tt tze tpl tt weq tt tt weq tt a2 tt tze tpl
       tt weq tt tze tpl tt weq tt tt weq wim tt a2
       tt tze tpl tt tt a1 mp mp
     $.
```

demo0.mm [ACCEPT] *Valid database per spec*

```
$( demo0.mm 1-Jan-04 $)

$(
                    PUBLIC DOMAIN DEDICATION

This file is placed in the public domain per the Creative Commons Public
Domain Dedication. http://creativecommons.org/licenses/publicdomain/
```

```
Norman Megill - email: nm at alum.mit.edu

$)


$( This file is the introductory formal system example described
   in Chapter 2 of the Meamath book. $)

$( Declare the constant symbols we will use $)
    $c 0 + = -> ( ) term wff |- $.
$( Declare the metavariables we will use $)
    $v t r s P Q $.
$( Specify properties of the metavariables $)
    tt $f term t $.
    tr $f term r $.
    ts $f term s $.
    wp $f wff P $.
    wq $f wff Q $.
$( Define "term" (part 1) $)
    tze $a term 0 $.
$( Define "term" (part 2) $)
    tpl $a term ( t + r ) $.
$( Define "wff" (part 1) $)
    weq $a wff t = r $.
$( Define "wff" (part 2) $)
    wim $a wff ( P -> Q ) $.
$( State axiom a1 $)
    a1 $a |- ( t = r -> ( t = s -> r = s ) ) $.
$( State axiom a2 $)
    a2 $a |- ( t + 0 ) = t $.
    ${
       min $e |- P $.
       maj $e |- ( P -> Q ) $.
$( Define the modus ponens inference rule $)
       mp  $a |- Q $.
    $}
$( Prove a theorem $)
    th1 $p |- t = t $=
  $( Here is its proof: $)
       tt tze tpl tt weq tt tt weq tt a2 tt tze tpl
       tt weq tt tze tpl tt weq tt tt weq wim tt a2
       tt tze tpl tt tt a1 mp mp
     $.
```

empty.mm [ACCEPT] *Empty database is valid*

issue129.mm [ACCEPT] *Valid database per spec*

```
$c A $.

ax-1 $a A A A A A $.

th1 $p A A A A A $= ax-1 $.

$( $t
    latexdef "A" as "\text{ A A A A A A A A A A }";
$)
```

issue134.mm [ACCEPT] *Valid database per spec*

```
$c A $.
ax1 $a A $.
```

local-var-good.mm [ACCEPT] *Valid variable scope per spec*

```
  $c formula $.
  $c A. $.
```

```
  $c setvar $.
  $v ph $.
  wph $f formula ph $.

  ${
    $v x $.
    $( Let ‘ x ‘ be an individual variable (temporary declaration). $)
    vx.wal $f setvar x $.
    $( Extend formula definition to include the universal quantifier (’for all’). $)
    wal $a formula A. x ph $.
  $}
```

min_found.mm [ACCEPT] *Valid database per spec*

```
$( A shortening found by the minimizer. The comparison with min_not_found.mm
   suggests that the minimizer’s ability to detect it depends on the presence
   of statement "A" in the steps of the minimized proof. The costant C and
   axiom ax-3 are not used here, but they are kept anyway to lessen the amount
   of differences with min_not_found.mm. $)

  $c A B C D $.

  ax-1 $a A $.
  ax-2 $a B $.
  ax-3 $a C $.

  ${
    in1.1 $e A $.
    in1 $a D $.
  $}

  ${
    in2.1 $e B $.
    in2.2 $e A $.
    in2.3 $e B $.
    in2.4 $e B $.
    in2.5 $e B $.
    in2 $a D $.
  $}

  $( Minimized proof, found automatically. (New usage is discouraged.) $)
  th1 $p D $= ( ax-1 in1 ) AB $.

  $( Non-minimized proof. $)
  th2 $p D $= ( ax-2 ax-1 in2 ) ABAAAC $.
```

min_not_found.mm [ACCEPT] *Valid database per spec*

```
$( A shortening not found by the minimizer. The comparison with min_found.mm
   suggests that the minimizer’s inability to detect it is due to the absence
   of statement "C" in any of the steps of the minimized proof. The
   shortening is detected if proveFloating is set to 1 in replaceStatement
   called by minimizeProof, which allows absent $e statements to be found. $)

  $c A B C D $.

  ax-1 $a A $.
  ax-2 $a B $.
  ax-3 $a C $.

  ${
    in1.1 $e A $.
    in1 $a D $.
  $}

  ${
    in2.1 $e B $.
    in2.2 $e C $.
```

```
    in2.3 $e B $.
    in2.4 $e B $.
    in2.5 $e B $.
    in2 $a D $.
  $}

  $( Minimized proof, not found automatically. (New usage is discouraged.) $)
  th1 $p D $= ( ax-1 in1 ) AB $.

  $( Non-minimized proof. $)
  th2 $p D $= ( ax-2 ax-3 in2 ) ABAAAC $.
```

nuniq-gram.mm [ACCEPT] *Valid database per spec (non-unique grammar allowed)*

```
$( Here we have two variable typecodes A and B, and one logical
   typecode |-, and the question is whether |- connects to A or B.
   It parses either way. This is actually an unambiguous formal system,
   because we require only that there are no two ways to derive the proof
   of the *same* syntax theorem, and "A *" and "B *" are different theorems. $)
$c A B |- * $. $v a b $.
va $f A a $.
vb $f B b $.
as $a A * $.
bs $a B * $.
ax $a |- * $.
```

test37_rpn_interleaving_mandatory_hyps.mm [ACCEPT] *Valid per RPN proof semantics*

```
$( Unit Test 37: RPN interleaving of $f and $e in mandatory hypotheses $)
$( Should reject: False - This documents correct behavior $)

$c wff |- -> ( ) $.
$v ph ps $.

${
  $( $f for ph $)
  wph $f wff ph $.

  $( $e using ph $)
  h1 $e |- ph $.

  $( $f for ps - AFTER the $e (interleaved!) $)
  wps $f wff ps $.

  $( Assertion: mandatory hyps in RPN order MUST be: wph, h1, wps $)
  $( NOT: wph, wps, h1 (which would be "all $f then $e") $)
  ax-test $a |- ( ph -> ps ) $.
$}

$( This test documents the CORRECT mandatory hypothesis order. $)
$( A verifier that incorrectly orders as "all $f, then all $e" would get: $)
$(   WRONG: wph, wps, h1 $)
$( The correct order per spec is appearance order: $)
$(   CORRECT: wph, h1, wps $)

$( To verify: use metamath.exe "show statement ax-test /full" $)
$( Expected output: "Its mandatory hypotheses in RPN order are:" $)
$(   wph $f wff ph $. $)
$(   h1 $e |- ph $. $)
$(   wps $f wff ps $. $)
```

test39_include_basic_outer.mm [ACCEPT] *Valid include at outermost scope*

```
$( Unit Test 39: Basic include - outer file $)
$( Should reject: False - legitimate include should work $)

$( Include provides constants, variables, and axioms $)
$[ ./helpers/test39_helper.mm $]
```

```
$( Use the included axiom to prove something $)
th1 $p |- ph $= wph ax-1 $.
```

test_complex_ehyp_syl2anc.mm [ACCEPT] *Valid proof per spec*

```
$( Minimized test case for syl2anc - tests complex essential hypothesis handling $)
$( Based on hol.mm by Mario Carneiro $)
$( This proof SHOULD verify correctly $)

$c term |- |= ( ) , $.
$v R S T A B $.

tr $f term R $.
ts $f term S $.
tt $f term T $.
ta $f term A $.
tb $f term B $.

$( Syntax: context pair $)
kct $a term ( A , B ) $.

$( Axioms with essential hypotheses $)
${
  ax-syl.1 $e |- R |= S $.
  ax-syl.2 $e |- S |= T $.
  ax-syl $a |- R |= T $.
$}

${
  ax-jca.1 $e |- R |= S $.
  ax-jca.2 $e |- R |= T $.
  ax-jca $a |- R |= ( S , T ) $.
$}

$( Derived theorems $)
${
  syl.1 $e |- R |= S $.
  syl.2 $e |- S |= T $.
  syl $p |- R |= T $=
    tr ts tt syl.1 syl.2 ax-syl $.
$}

${
  jca.1 $e |- R |= S $.
  jca.2 $e |- R |= T $.
  jca $p |- R |= ( S , T ) $=
    tr ts tt jca.1 jca.2 ax-jca $.
$}

${
  syl2anc.1 $e |- R |= S $.
  syl2anc.2 $e |- R |= T $.
  syl2anc.3 $e |- ( S , T ) |= A $.
  syl2anc $p |- R |= A $=
    tr ts tt kct ta tr ts tt syl2anc.1 syl2anc.2 jca syl2anc.3 syl $.
$}
```

test59_f_type_global_conflict.mm [ACCEPT] *SPEC DIVERGENCE: L156-158 says REJECT, all impls ACCEPT*

```
$( test59_f_type_global_conflict.mm - ACCEPT (SPEC DIVERGENCE!)
   Spec 4.1.3 L156-158 says:
   "The type declared by a $f statement for a given label is global even if
   the variable is not (e.g., a database may not have wff P in one local
   scope and class P in another)."

   HOWEVER: All three verifiers (metamath.exe, metamath-knife, mmverify.py)
   ACCEPT having the same variable with different types in different scopes!
```

```
   This is documented as a SPEC DIVERGENCE: the spec says REJECT, but
   all implementations ACCEPT. The convention in set.mm is to never do this,
   so it's never been caught.

   For our test suite, we follow the IMPLEMENTATIONS (ACCEPT), not the spec.
$)

$c wff class |- $.

$( First scope: x is wff $)
${
  $v x $.
  wx $f wff x $.
  ax1 $a wff x $.
$}

$( Second scope: x is class - spec says REJECT, implementations say ACCEPT $)
${
  $v x $.
  cx $f class x $.
  ax2 $a class x $.
$}
```

test60_empty_block.mm [ACCEPT] *EBNF: empty block \${ \$} is valid*

```
$( test60_empty_block.mm - ACCEPT
   Tests EBNF: block ::= '${' stmt* '$}'
   Zero statements in a block is valid.

   Empty blocks between valid proofs should be accepted.
$)

$c wff |- $.
$v p q $.
wp $f wff p $.
wq $f wff q $.

$( First proof $)
th1 $a wff p $.

$( Empty block - should be valid $)
${ $}

$( Another proof after empty block $)
th2 $a wff q $.

$( Nested empty blocks $)
${
  ${ $}
  ${ $}
$}

$( Final proof $)
th3 $p wff p $= wp $.
```

test57_compressed_z_minimal.mm [ACCEPT] *Valid compressed proof with Z*

```
$( Minimal test of compressed proof with Z (save) marker $)
$( This is designed to be the simplest possible Z test that should pass $)

$c wff ( ) -> $.
$v ph ps $.
wph $f wff ph $.
wps $f wff ps $.

$( Binary connective - implication $)
wi $a wff ( ph -> ps ) $.
```

```
$( Simple proof using Z:
   Prove: wff ( ( ph -> ps ) -> ( ph -> ps ) )

   Labels: wph=0, wps=1, wi=2

   Proof steps:
   A = 0 = push wph -> stack: [wff ph]
   B = 1 = push wps -> stack: [wff ph, wff ps]
   C = 2 = apply wi -> stack: [wff ( ph -> ps )]
   Z = -1 = save top -> saved: [wff ( ph -> ps )], stack: [wff ( ph -> ps )]
   D = 3 = recall saved[0] -> stack: [wff ( ph -> ps ), wff ( ph -> ps )]
   C = 2 = apply wi -> stack: [wff ( ( ph -> ps ) -> ( ph -> ps ) )]

   Compressed: ABCZDC
$)
th1 $p wff ( ( ph -> ps ) -> ( ph -> ps ) ) $= ( wi ) ABCZDC $.
```

test_compressed_simple.mm [ACCEPT] *Valid compressed proof*

```
$( Minimal test for compressed proof $)

$c term |- |= $.
$v R S T $.

tR $f term R $.
tS $f term S $.
tT $f term T $.

${
  syl.1 $e |- R |= S $.
  syl.2 $e |- S |= T $.
  ax-syl $a |- R |= T $.
$}

${
  th.1 $e |- R |= S $.
  th.2 $e |- S |= T $.
  $( Uncompressed proof - should work $)
  th $p |- R |= T $= tR tS tT th.1 th.2 ax-syl $.
$}
```

test_dv_pairwise.mm [ACCEPT] *Valid pairwise \$d*

```
$( Test for DV pairwise expansion: $d x y z $. should create pairs (x,y), (x,z), (y,z) $)

$c wff |- $.
$v x y z $.

wx $f wff x $.
wy $f wff y $.
wz $f wff z $.

${
  $d x y z $.

  $( Axiom requiring x,y disjoint $)
  ax-xy.1 $e |- x $.
  ax-xy $a |- y $.
$}

${
  $d x y z $.

  $( Axiom requiring x,z disjoint $)
  ax-xz.1 $e |- x $.
  ax-xz $a |- z $.
$}
```

```
${
  $d x y z $.

  $( Axiom requiring y,z disjoint - THIS IS THE KEY TEST $)
  ax-yz.1 $e |- y $.
  ax-yz $a |- z $.
$}

$( Test theorem - uses all three DV pairs $)
${
  $d x y z $.
  th.1 $e |- x $.
  th $p |- z $= wx wz th.1 ax-xz $.
$}
```

test_dv_yz_required.mm [ACCEPT] *Valid \$d test*

```
$( Test for DV pairwise expansion: $d x y z $. should create pair (y,z)
   This test REQUIRES (y,z) to be disjoint - will FAIL if pairwise expansion broken $)

$c wff |- $.
$v x y z $.

wx $f wff x $.
wy $f wff y $.
wz $f wff z $.

$( Key axiom: requires y and z to be disjoint $)
${
  $d y z $.
  ax-yz.1 $e |- y $.
  ax-yz $a |- z $.
$}

$( Theorem using multi-variable $d declaration
   The $d x y z $. MUST expand to include (y,z) for this to verify! $)
${
  $d x y z $.
  th.1 $e |- y $.
  th $p |- z $= wy wz th.1 ax-yz $.
$}
```

test_stack_fhyps.mm [ACCEPT] *Valid stack test*

```
$( Test for floating hypothesis consumption from stack $)

$c wff |- term |= $.
$v R S T $.

wR $f term R $.
wS $f term S $.
wT $f term T $.

${
  ax-syl.1 $e |- R |= S $.
  ax-syl.2 $e |- S |= T $.
  $( Syllogism: if R implies S and S implies T, then R implies T $)
  ax-syl $a |- R |= T $.
$}

${
  syl.1 $e |- R |= S $.
  syl.2 $e |- S |= T $.
  $( Proof using ax-syl - should need 5 items on stack:
     3 floating hyps (term R, term S, term T) + 2 essential hyps $)
  syl $p |- R |= T $= wR wS wT syl.1 syl.2 ax-syl $.
$}
```

test_stack_simple.mm [ACCEPT] *Valid stack test*

```
$( Simple test for stack operations $)

$c wff |- $.
$v x y $.

wx $f wff x $.
wy $f wff y $.

${
  ax $a |- x $.
  th $p |- y $= wy ax $.
$}
```

C.35 Miscellaneous

test41_include_multiple_main.mm [ACCEPT] *Multiple valid includes at outermost scope*

```
$( Unit Test 41: Multiple includes $)
$( Should reject: False - multiple includes should work $)

$c wff |- $.

$( Include file A $)
$[ ./helpers/test41_helper_a.mm $]

$( Include file B $)
$[ ./helpers/test41_helper_b.mm $]

$( Use content from both includes $)
th-a $p |- var-a $= fa ax-a $.
th-b $p |- var-b $= fb ax-b $.
```

test70_djvars_before_float.mm [ACCEPT] *Metamath accepts \$d before corresponding \$f declarations*

```
$( Test 70: $d may appear before corresponding $f hypotheses. $)

$c wff |- $.
$v x y $.
$d x y $.

hx $f wff x $.
hy $f wff y $.

a1 $a |- x $.
```

test58_compressed_ehyp_z.mm [ACCEPT] *compressed proof with essential hyps and Z markers*

```
$( Minimal test: compressed proof with essential hypotheses AND Z marker $)
$( Based on demo0.mm structure - tests the combination that fails in big-unifier.mm $)
$(
  This test case is designed to isolate the bug where:
  - Compressed proofs work (test57 passes)
  - Normal proofs with e-hyps work (demo0.mm passes)
  - BUT compressed proofs with e-hyps + Z marker FAIL (big-unifier.mm fails)

  This file contains both normal and compressed versions of the same proof,
  allowing direct comparison of verification behavior.
$)

$c 0 + = -> ( ) term wff |- $.
$v t r s P Q $.

tt $f term t $.
tr $f term r $.
```

```
ts $f term s $.
wp $f wff P $.
wq $f wff Q $.

$( Define term $)
tze $a term 0 $.
tpl $a term ( t + r ) $.

$( Define wff $)
weq $a wff t = r $.
wim $a wff ( P -> Q ) $.

$( Axioms $)
a1 $a |- ( t = r -> ( t = s -> r = s ) ) $.
a2 $a |- ( t + 0 ) = t $.

$( Modus ponens - has essential hypotheses $)
${
  min $e |- P $.
  maj $e |- ( P -> Q ) $.
  mp $a |- Q $.
$}

$( Normal proof of |- t = t - should work $)
th1_normal $p |- t = t $=
  tt tze tpl tt weq tt tt weq tt a2 tt tze tpl tt weq tt tze tpl tt weq tt tt
  weq wim tt a2 tt tze tpl tt tt a1 mp mp $.

$(
  Compressed proof of |- t = t generated by Metamath program.
  This proof:
  1. Uses compressed format
  2. Uses modus ponens (mp) which has essential hypotheses
  3. Uses Z marker to save/recall intermediate results

  Labels:
    A=tt (mandatory f-hyp)
    B=tze, C=tpl, D=weq, E=a2, F=wim, G=a1, H=mp (explicit labels)

  Proof string: ABCZADZAADZAEZJJKFLIAAGHH

  The Z at position 4 saves something, then J (index 9) recalls it.
  This combines all three ingredients that cause big-unifier.mm to fail.
$)
th1_compressed $p |- t = t $=
  ( tze tpl weq a2 wim a1 mp ) ABCZADZAADZAEZJJKFLIAAGHH $.
```

test65_compressed_assertion_fhyp_order.mm [ACCEPT] *Spec AppB: compressed proofs with scoped f-hyps*

```
$( Unit Test 65: Compressed proof with 3-variable axiom $)
$( Tests: Correctly using an axiom with 3 f-hyps in a compressed proof.
   ax-3var has mandatory hyps: wff ph, wff ps, wff ch, |- ph, |- ps
   We prove |- ps by calling ax-3var with substitution ch -> ps
   IMPORTANT: Everything wrapped in outer ${ $} to avoid top-level $e $)
$( Should accept: True $)

$c wff |- $.
$v ph ps ch $.

${
  wp $f wff ph $.
  wq $f wff ps $.
  wc $f wff ch $.

  $( Axiom: from |- ph and |- ps, conclude |- ch $)
  ax.1 $e |- ph $.
  ax.2 $e |- ps $.
```

```
  ax-3var $a |- ch $.

  $( Theorem: prove |- ps by using ax-3var with ch -> ps $)
  $( Mandatory hyps of th: wp(A), wq(B), ax.1(C), ax.2(D), th.1(E), th.2(F)
     Note: ax.1 and ax.2 are mandatory because they're used in the proof!
     Label list: (ax-3var) -> G = ax-3var
     Proof: A B B E F G = push wff ph, wff ps, wff ps, |- ph, |- ps, call ax-3var
     ax-3var with {ph->ph, ps->ps, ch->ps} yields |- ps $)
  th.1 $e |- ph $.
  th.2 $e |- ps $.
  th $p |- ps $= ( ax-3var ) ABBEFG $.
$}
```

test66_compressed_mand_hyp_order.mm [ACCEPT] *Spec AppB: mandatory hypothesis ordering*

```
$( Unit Test 66: Compressed proof - mandatory hypothesis ordering $)
$( Tests: Mandatory hyps are f-hyps (in order) + e-hyps (in order) $)
$( Should accept: True $)

$c wff |- $.
$v ph ps $.
wp $f wff ph $.
wq $f wff ps $.

${
  $( Axiom with 2 essential hyps $)
  min.1 $e |- ph $.
  min.2 $e |- ps $.
  ax-min $a |- ph $.
$}

${
  $( Proof using compressed format $)
  $( Mandatory hyps: wp(A), wq(B), th.1(C), th.2(D). Label list: E=ax-min $)
  th.1 $e |- ph $.
  th.2 $e |- ps $.
  th $p |- ph $= ( ax-min ) ABCDE $.
$}
```

test67_toplevel_essential.mm [ACCEPT] *Spec allows top-level \$e (knife non-compliant)*

```
$( Unit Test 67: Top-level $e statements $)
$( Tests: Essential hypotheses at the outermost scope level
   Spec allows this - no restriction in EBNF or spec text.
   metamath-knife incorrectly REJECTS this (non-compliant).
   Official Metamath correctly ACCEPTS this. $)
$( Should accept: True $)

$c wff |- $.
$v ph ps $.
wp $f wff ph $.
wq $f wff ps $.

$( Top-level essential hypothesis - not inside any ${ $} block $)
hyp1 $e |- ph $.

$( This assertion has hyp1 as a mandatory hypothesis $)
ax1 $a |- ps $.
```

test_axmp_z.mm [REJECT] *Broken compressed proof - both verifiers reject*

```
$( Minimal test: ax-mp with essential hypotheses and Z marker $)

$c wff |- e ( , ) $.
$v x y $.

wx $f wff x $.
wy $f wff y $.
```

```
$( Binary connective $)
wi $a wff e ( x , y ) $.

$( Modus ponens with essential hypotheses $)
${
  ax-mp.1 $e |- x $.
  ax-mp.2 $e |- e ( x , y ) $.
  ax-mp $a |- y $.
$}

$( A simple axiom to use $)
ax1 $a |- e ( x , e ( y , x ) ) $.

$(
  Compressed proof using Z:
  Goal: |- e ( x , e ( y , x ) )

  Labels: wx=0, wy=1, wi=2, ax-mp=3, ax1=4

  Proof steps for normal proof:
  wx wy ax1 => |- e ( x , e ( y , x ) )

  As compressed: ( wi ax-mp ax1 ) ABC
    A=0=wx, B=1=wy, C=2=ax1
$)
th1 $p |- e ( x , e ( y , x ) ) $= ( wi ax-mp ax1 ) ABC $.
```

test_compressed_syl.mm [REJECT] *Broken compressed proof - both verifiers reject*

```
$( Minimal test for compressed proof with multiple floating hypotheses $)

$c term |- |= $.
$v R S T $.

tR $f term R $.
tS $f term S $.
tT $f term T $.

${
  syl.1 $e |- R |= S $.
  syl.2 $e |- S |= T $.
  $( Syllogism axiom $)
  ax-syl $a |- R |= T $.
$}

${
  th.1 $e |- R |= S $.
  th.2 $e |- S |= T $.
  $( Theorem using compressed proof - should push tR, tS, tT before ax-syl $)
  th $p |- R |= T $= ( ax-syl ) BCA $.
$}
```

test_dv_multi.mm [REJECT] *Comment says INVALID PROOF - should be rejected*

```
$( Test for multi-variable $d declarations - INVALID PROOF (should be rejected) $)

$c wff |- $.
$v x y z $.

wx $f wff x $.
wy $f wff y $.
wz $f wff z $.

${
  $d x y z $.
  ax $a |- x $.

  ${
```

```
    th.1 $e |- y $.
    th $p |- z $= wy wz ax th.1 $.
  $}
$}
```

C.36 Large Test Files (Summary Only)

The following test files exceed 70 lines and are summarized rather than listed verbatim.

File	Verdict	Lines	Spec Rule
`big-unifier-bad1.mm`	REJECT	100	4.2.1: unification/substitution failure
`big-unifier-bad2.mm`	REJECT	100	4.2.1: unification/substitution failure
`big-unifier-bad3.mm`	REJECT	100	4.2.1: unification/substitution failure
`big-unifier.mm`	ACCEPT	341	Valid database per spec
`miu.mm`	ACCEPT	131	Valid database per spec
`peano-fixed.mm`	ACCEPT	859	Valid database per spec

References

[1] N. Megill and D. A. Wheeler, *Metamath: A Computer Language for Mathematical Proofs*, Lulu Press, 2019. Available at http://us.metamath.org/downloads/metamath.pdf.

[2] M. Carneiro, "Metamath Zero: Designing a Theorem Prover Prover," *arXiv preprint arXiv:1910.10703*, 2019.

[3] L. de Moura and S. Ullrich, "The Lean 4 Theorem Prover and Programming Language," in *CADE-28*, LNCS vol. 12699, Springer, 2021, pp. 625–635.

[4] Lean Community, "Batteries: The Lean 4 Standard Library Extension," https://github.com/leanprover-community/batteries, 2024.

Table 3: Complete file inventory (54,130 lines total).

File	Lines
Metamath/AllM.lean	128
Metamath/ArrayListExt.lean	818
Metamath/Bridge/Basics.lean	251
Metamath/Bridge.lean	91
Metamath/ByteSliceCompat.lean	84
Metamath/CounterexampleInsertError.lean	102
Metamath/DBCaseAnalysis.lean	2,006
Metamath/DBLemmas.lean	218
Metamath/DeclarativeSpec.lean	640
Metamath/ErrorCodeSemantics.lean	490
Metamath/FrontendAudit.lean	263
Metamath/FrontendBridge.lean	544
Metamath/FrontendCertified.lean	130
Metamath/GuardCombinators.lean	182
Metamath/HashMapLemmas.lean	683
Metamath/KernelClean.lean	11,395
Metamath/KernelExtras.lean	238
Metamath/LoopInvariant.lean	81
Metamath/ParserAnyModeEquivalence.lean	270
Metamath/ParserBasics.lean	158
Metamath/ParserCorrectnessCore.lean	23
Metamath/ParserCorrectness.lean	1,906
Metamath/ParserEquivalenceExamples.lean	74
Metamath/ParserEquivalence.lean	492
Metamath/ParserInvariants.lean	618
Metamath/ParserInvariantsStep1.lean	178
Metamath/ParserLoopInduction.lean	1,896
Metamath/ParserOperations.lean	9,804
Metamath/ParserProofs.lean	1,772
Metamath/ParserSoundnessDemo.lean	150
Metamath/PrefixProvenance.lean	2,697
Metamath/PrefixTraceCompressed.lean	1,045
Metamath/PrefixWitnessCheckBytes.lean	2,125
Metamath/Spec/Bridge.lean	1,344
Metamath/Spec/Core.lean	234
Metamath/Spec/Equivalence.lean	3,540
Metamath/Spec/Frontend.lean	183
Metamath/Spec.lean	5
Metamath/Spec/Operational.lean	270
Metamath/Spec/Semantic.lean	10
Metamath/Tests/CliErrorCodeFormat.lean	42
Metamath/Tests/ParserInvariantTests.lean	191
Metamath/ValidateDB.lean	212
Metamath/Verify.lean	5,601
Metamath/WellFormedness.lean	921
Metamath/ZipperTest.lean	25